DATA STRUCTURES & ALGORITHMS · INTERVIEW MASTERY SERIES

Graph Data Structures The Complete FAANG Interview Guide

Every graph type, every representation, every core algorithm — with visual diagrams, worked examples, and the exact interview triggers senior engineers look for.

12 Graph Types | 4 Representations | 15+ Core Algorithms | 50 Curated Problems | Pattern Cheat Sheet

Print-ready · A4 · Study offline or export to PDF

START HERE

Table of Contents

HOW TO USE THIS GUIDE

Read sections 1–5 to lock in fundamentals, sections 6–13 to master every core algorithm asked in FAANG loops, section 14 as a rapid-recall cheat sheet the night before an interview, and section 15 to drill real problems in increasing difficulty.

01 What Is a Graph Data Structure?

A **graph** is a non-linear data structure made of a set of **vertices** (nodes) connected by **edges** (connections). Unlike arrays, lists, or trees, a graph has no fixed hierarchy — any vertex can connect to any other vertex, which makes it the most general-purpose data structure for modeling relationships.

Why Graphs Are Everywhere

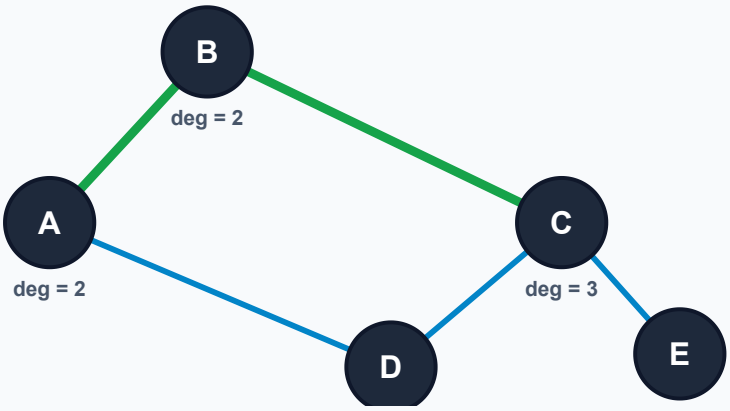
- **Social networks** — people as vertices, friendships/follows as edges.
- **Maps & routing** — intersections as vertices, roads as weighted edges.
- **Dependency resolution** — build systems, package managers, course prerequisites.
- **Web & networks** — pages/routers as vertices, links/connections as edges.
- **State machines & game boards** — states/cells as vertices, valid moves as edges.

Core Terminology You Must Know Cold

- **Vertex / Node (V):** a single entity in the graph.
- **Edge (E):** a connection between two vertices.
- **Adjacent:** two vertices directly connected by an edge.
- **Degree:** number of edges touching a vertex (split into *in-degree* / *out-degree* for directed graphs).
- **Path:** a sequence of vertices connected by edges, no repeated vertices.
- **Walk:** like a path, but vertices/edges may repeat.
- **Order / Size:** |V| = order (vertex count), |E| = size (edge count).

Anatomy of a Graph

LABELED EXAMPLE



*Vertices A–E connected by edges. The highlighted green route A→B→C is a **path**. Vertex C has **degree 3** (adjacent to B, D and E) — the most "central" node here.*

🎯 **FAANG INTERVIEW TRIGGER**

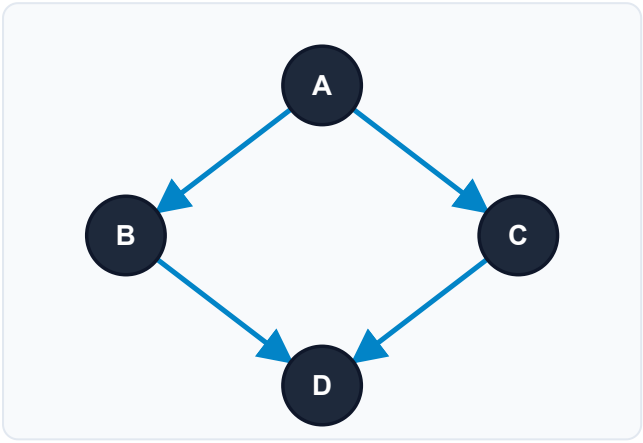
A huge fraction of "array" or "grid" interview problems are secretly graph problems in disguise (matrix neighbors = implicit edges, word transformations = implicit edges). The #1 skill interviewers test first is whether you can **recognize a graph** hiding inside a problem statement before you recognize it's asking for DFS/BFS/Union-Find.

02 Classifying Graphs: Direction & Weight

Every graph can be classified along several **independent axes**. The first two — edge direction and edge weight — determine which traversal and shortest-path algorithms are even legal to use. The diagrams below reuse the *same four vertices* so you can see exactly what changes on each axis.

Directed Graph

DIGRAPH

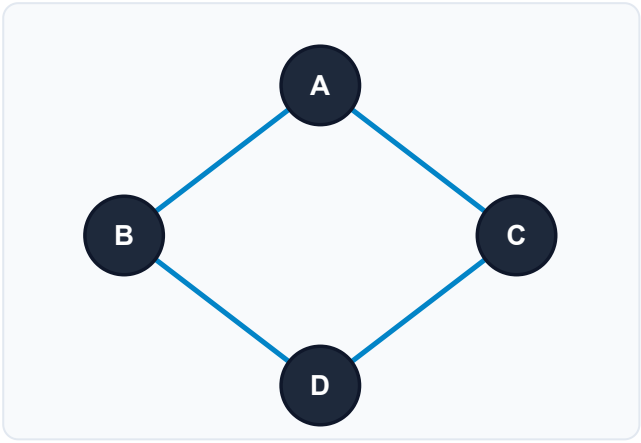


Edges have a specific direction — represented as an ordered pair (u, v). You may only traverse from u → v, never the reverse, unless a matching (v, u) edge also exists.

- Real-world: task dependencies, one-way streets, Twitter "follows", web page links.
- Traversal allowed only along the arrow direction.

> 🎯 **FAANG TRIGGER**
> Words like "prerequisite", "depends on", "can reach" → directed graph. Expect **Topological Sort** or DFS-with-recursion-stack cycle detection.

Undirected Graph



Edges have no direction — represented as an unordered pair {u, v}. Traversal is always allowed in both directions.

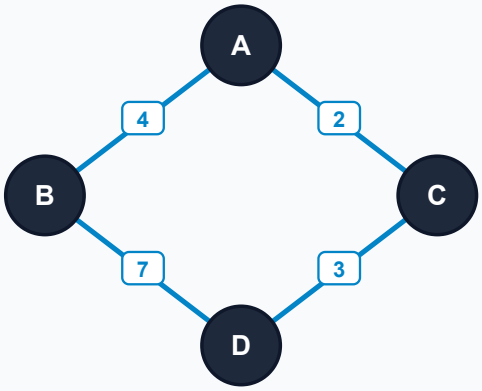
- Real-world: friendships, physical cables/roads, mutual connections.
- Every edge is logically two directed edges (u→v and v→u).

> 🎯 **FAANG TRIGGER**
> "How many groups/islands/clusters are connected?" on an undirected graph → reach for **Union-Find (DSU)** first; it beats DFS/BFS on follow-up "add edges dynamically" twists.

FEATURE	DIRECTED GRAPH	UNDIRECTED GRAPH
Edge direction	Has a direction	No direction
Representation	Ordered pair (u, v)	Unordered pair {u, v}
Traversal	Only along edge direction	Both directions
Relationship type	One-way	Two-way
Example	One-way roads, task dependencies	Friendships, two-way roads
Typical cycle check	DFS + recursion stack	DFS parent-tracking or Union-Find
Adjacency matrix	Not necessarily symmetric	Always symmetric

02 Classifying Graphs: Weight

Weighted Graph

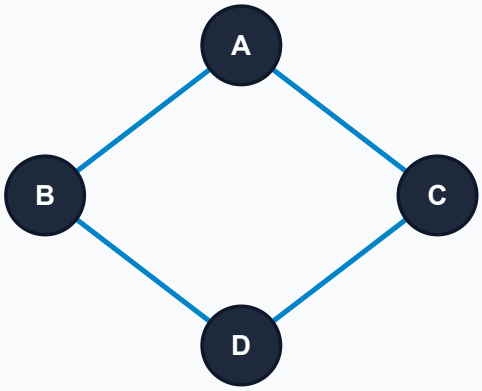


Each edge carries an associated **weight or cost** (distance, time, price, bandwidth). The "shortest path" becomes "minimum total weight path", not "fewest edges".

- Real-world: flight prices, road distances, network latency.
- Weights may be non-negative only (Dijkstra) or include negatives (Bellman-Ford).

> 🎯 **FAANG TRIGGER**
> Problem mentions "minimum cost", "cheapest", "time to reach" → **Dijkstra** (non-negative) or **Bellman-Ford** (negatives / detect negative cycle).

Unweighted Graph



Edges carry **no weight** — every edge implicitly costs exactly 1 hop. "Shortest path" simply means "fewest edges traversed".

- Real-world: unweighted social graphs, grid mazes with uniform step cost.
- Solved optimally with plain BFS — no priority queue needed.

> 🎯 **FAANG TRIGGER**
> "Fewest steps", "minimum moves", "shortest transformation sequence" → plain **BFS**. Using Dijkstra here is a correctness-preserving but complexity-wasteful red flag interviewers notice immediately.

MENTAL MODEL

Direction and weight are **orthogonal** — a graph can be directed+weighted (flight routes with prices), undirected+unweighted (simple friendship graph), or any other combination. Always identify both axes before picking an algorithm.

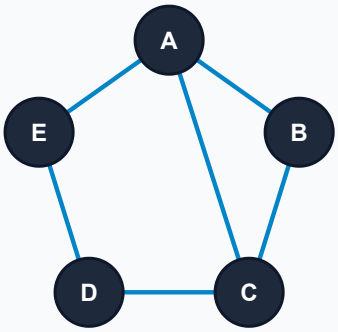
COMMON MISTAKE

Treating an unweighted graph's adjacency as "weight 0" instead of "weight 1 per hop" is a classic off-by-one bug that breaks BFS-based shortest path counts. Each edge traversed always adds exactly 1 to the hop count.

03 Classifying Graphs: Connectivity

Connectivity asks a single question: **can every vertex reach every other vertex?** This determines whether you need one traversal call or several, and whether Union-Find will report one set or many.

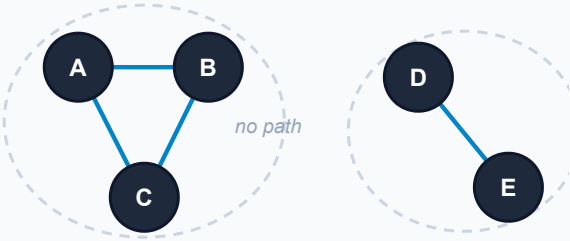
Connected Graph



A path exists between **every pair of vertices**. Starting a single DFS/BFS from any vertex visits the entire graph.

> 🎯 **FAANG TRIGGER**
>
> "Is the graph fully connected?" → one traversal call; if the visited-count equals |V|, it's connected. Also the base case Union-Find reduces to **exactly one** set.

Disconnected Graph

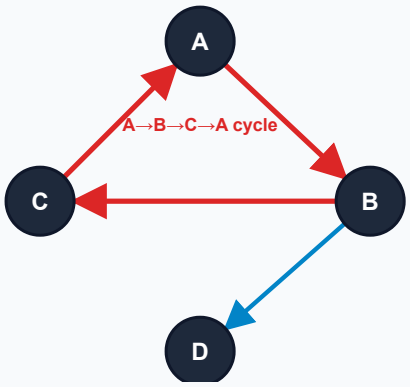


One or more vertices are **unreachable** from others — the graph splits into multiple connected components.

> 🎯 **FAANG TRIGGER**
>
> "Count the number of islands / provinces / groups" → loop over all vertices, run DFS/BFS (or Union-Find) from every unvisited one, and count how many traversals you start. This is the **Multi-Source / Component Counting** pattern.

Classifying Graphs: Cycles

Cyclic Graph

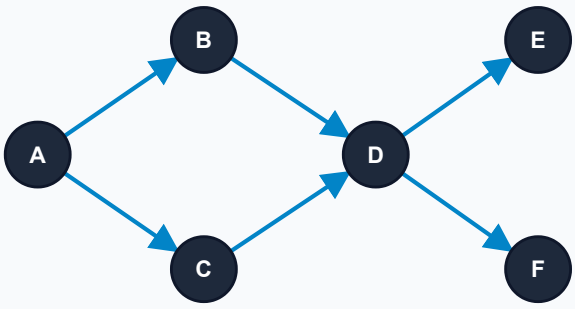


Contains at least one **cycle** — a path that starts and ends at the same vertex without repeating edges/vertices in between (A→B→C→A here, shown in red).

> 🎯 **FAANG TRIGGER**
>
> Deadlock detection, "can this ever loop forever?" → DFS with a **recursion stack** (directed) or parent-tracking (undirected) to detect back-edges.

Directed Acyclic Graph

DAG



A **directed** graph with **no cycles**. Every DAG has at least one valid topological ordering of its vertices.

> 🎯 **FAANG TRIGGER**
>
> "Course schedule", "build order", "task dependencies" → this is a DAG. Use **Topological Sort** (Kahn's BFS or DFS post-order). If a cycle is found, no valid order exists.

QUICK DEFINITIONS

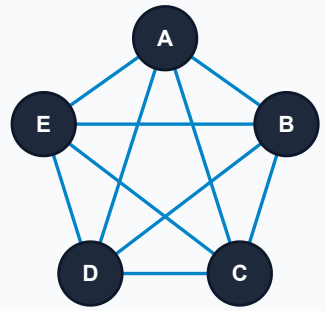
A **cycle** begins and ends at the same vertex, visiting each intermediate vertex only once, and can exist in both directed and undirected graphs. A **tree** is simply a connected, acyclic, undirected graph with exactly V−1 edges — the "simplest possible" connected structure.

04 Structural Special Cases

Beyond direction, weight, connectivity and cycles, three **structural shapes** come up constantly in interviews because each unlocks a different algorithmic shortcut.

Complete Graph

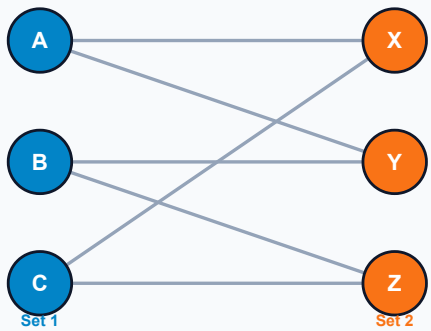
K_N



Every pair of distinct vertices is connected by an edge. A complete graph on n vertices (K_n) has exactly **n(n–1)/2** edges (undirected) — the maximum possible.

> **FAANG TRIGGER**
>
> Signals a **dense** graph → prefer adjacency matrix over list. Also the base case for clique-detection and Traveling-Salesman-style hard problems.

Bipartite Graph

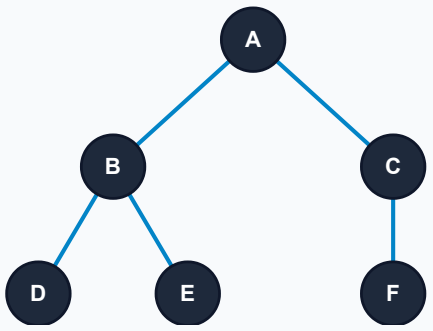


Vertices split into two disjoint sets (blue / orange) such that **every edge connects the two sets** — never two vertices from the same set.

> **FAANG TRIGGER**
>
> "Can you split people/items into two groups given constraints?" → run a **BFS/DFS 2-coloring** check. A graph is bipartite **iff it has no odd-length cycle**. Classic problems: "Is Graph Bipartite?", "Possible Bipartition".

Tree

CONNECTED + ACYCLIC



A tree is simply a graph that is **connected**, **acyclic**, and **undirected**, with exactly **V – 1 edges** for V vertices.

- Exactly one unique path exists between any two vertices.
- Adding any single edge creates exactly one cycle.
- Removing any single edge disconnects the tree into two components.

> **FAANG TRIGGER**
>
> If the prompt guarantees "the input is a tree", you can safely drop cycle-handling logic (just avoid revisiting the parent). "Redundant Connection" (find the extra edge that creates a cycle) is solved directly with **Union-Find**.

Density Spectrum — Why This Matters Next

SPARSE — DENSE

SHAPE	EDGE COUNT	BEST REPRESENTATION
Tree	V – 1 (sparsest connected graph)	Adjacency List
Typical connected graph	Between V–1 and V(V–1)/2	Adjacency List (usually)
Complete graph K_n	V(V–1)/2 (densest possible)	Adjacency Matrix

This density trade-off is exactly what determines which **representation** you should reach for — covered next.

05 Graph Representations

The same graph can be stored in memory in several ways. Interviewers care deeply about this choice because it changes the time complexity of every algorithm you write afterward. All four diagrams below encode the identical reference graph: **edges A–B, A–C, B–C, C–D**.

Adjacency Matrix

DENSE

V	A	B	C	D
A	0	1	1	0
B	1	0	1	0
C	1	1	0	1
D	0	0	1	0

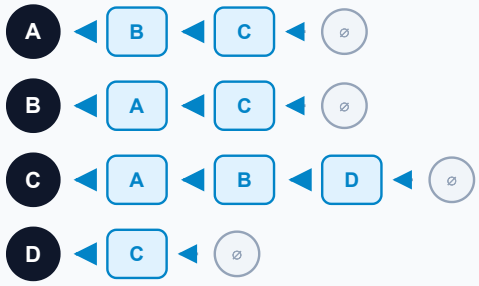
A **V × V** 2D array where cell (i, j) stores 1 (or the edge weight) if an edge exists between i and j, else 0.

- O(1) edge existence lookup — just index the array.
- O(V²) space regardless of how few edges exist.
- Symmetric across the diagonal for undirected graphs.

> 🎯 **FAANG TRIGGER**
>
> Choose adjacency matrix when V is small (≤ a few thousand), the graph is **dense**, or you need O(1) "is there an edge between i and j" checks (e.g. Floyd-Warshall requires it).

Adjacency List

SPARSE



Each vertex keeps a **list of its direct neighbors**. Only existing connections are stored, so memory scales with actual edges, not V².

- O(V + E) total memory — the most common representation in interviews.
- Efficient to enumerate a vertex's neighbors — exactly what DFS/BFS need.
- Typically implemented as `Map<Vertex, List<Vertex>>` or array-of-lists.

> 🎯 **FAANG TRIGGER**
>
> This is the **default choice** in 90% of interview solutions — almost every DFS/BFS/Dijkstra template starts with `graph = defaultdict(list)`. Reach for the matrix only when you have a specific reason.

05 Graph Representations — Continued

Edge List

(A, B) (A, C) (B, C) (C, D)

The graph stored as a flat **list of edges**, each a pair (or tuple) of vertices — optionally with a weight, e.g. (A, B, 4).

- Simplest possible representation — no per-vertex structure at all.
- Ideal when you need to **sort** or **process** edges independent of vertices.

> 🎯 **FAANG TRIGGER**
> Kruskal's MST algorithm needs edges **sorted by weight** — always start from an edge list there. Also the natural input format for Union-Find-based problems.

Incidence Matrix

	e1	e2	e3	e4
A	1	1	0	0
B	1	0	1	0
C	0	1	1	1
D	0	0	0	1

A **V × E** matrix where rows are vertices, columns are edges, and each cell marks whether a vertex is an endpoint of that edge.

- Common in formal graph theory and circuit/flow-network analysis.
- Rarely hand-implemented in interviews, but useful to recognize.

> 🎯 **FAANG TRIGGER**
> Rare in coding rounds; occasionally referenced in **systems/EE-adjacent** questions (circuit graphs, flow networks). Know the definition, don't over-invest in implementing it.

FEATURE	ADJACENCY MATRIX	ADJACENCY LIST	EDGE LIST	INCIDENCE MATRIX
Space	$O(V^2)$	O(V + E)	O(E)	O(V × E)
Edge lookup (u,v)	O(1)	O(degree of u)	O(E)	O(E)
Traverse neighbors	O(V)	O(degree of vertex)	O(E)	O(E)
Add an edge	O(1)	O(1) average	O(1)	O(V)
Remove an edge	O(1)	O(degree of vertex)	O(E)	O(V)
Best suited for	Dense graphs	Sparse graphs (default choice)	Kruskal's, sorting edges	Theory / flow networks

INTERVIEW DEFAULT

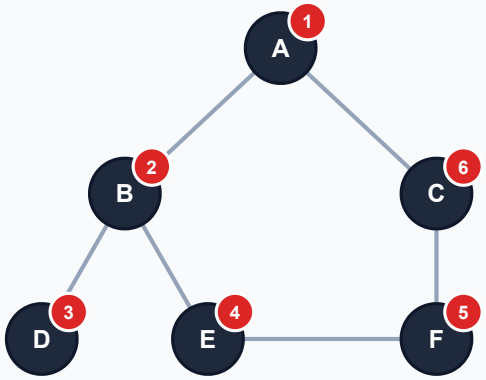
When a problem doesn't specify, build an **adjacency list** (hash map of lists, or array of lists when vertices are 0..V-1 integers). It's the right choice for the overwhelming majority of DFS/BFS/shortest-path/MST problems.

06 Traversals: DFS vs BFS

Both algorithms visit every reachable vertex exactly once — they differ only in **order** and the **data structure** driving that order. The same graph below (edges A-B, A-C, B-D, B-E, C-F, E-F) is traversed from A by each algorithm; the orange/green badges show visit order.

Depth First Search

STACK / RECURSION



Order: A → B → D (dead end, backtrack) → E → F → C

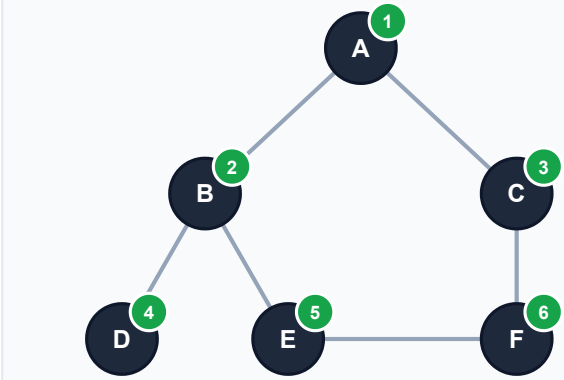
- Explores as **deep** as possible before backtracking.
- Implemented via explicit stack, or implicitly via recursion.
- Used for: cycle detection, topological sort, connected components, path existence.

FAANG TRIGGER

"Explore all paths", "does a path exist", "connected components", backtracking-flavored problems → DFS is almost always simpler to write recursively.

Breadth First Search

QUEUE



Order: A → B, C (level 1) → D, E, F (level 2)

- Explores **level by level**, visiting all neighbors before going deeper.
- Implemented with an explicit queue (FIFO).
- Used for: shortest path in unweighted graphs, level-order problems, multi-source spread.

FAANG TRIGGER

"Shortest path", "minimum steps", "fewest hops", "level order" on an **unweighted** graph → BFS, guaranteed optimal the first time a vertex is reached.

DFS Template (Recursive)

```
def dfs(node, visited, graph):
    if node in visited:
        return
    visited.add(node)
    # process(node) here

    for neighbor in graph[node]:
        dfs(neighbor, visited, graph)

# Driver — handles disconnected graphs
too
visited = set()
for v in all_vertices:
    if v not in visited:
        dfs(v, visited, graph)
```

BFS Template (Queue)

```
from collections import deque

def bfs(start, graph):
    visited = {start}
    queue = deque([start])
    dist = {start: 0}

    while queue:
        node = queue.popleft()
        # process(node) here

        for neighbor in graph[node]:
            if neighbor not in visited:
                visited.add(neighbor)
                dist[neighbor] =
dist[node] + 1
                queue.append(neighbor)
    return dist
```

06 DFS vs BFS — Head to Head

FEATURE	DFS	BFS
Traversal order	Goes deep before backtracking	Visits level by level
Data structure	Stack (or recursion call stack)	Queue
Shortest path (unweighted)	Not guaranteed	Guaranteed
Memory usage	Usually lower (O(depth))	Usually higher (O(width))
Common applications	Cycle detection, topological sort, backtracking	Shortest path, level order, multi-source spread

REPRESENTATION	DFS TIME	BFS TIME
Adjacency List	O(V + E)	O(V + E)
Adjacency Matrix	$O(V^2)$	$O(V^2)$

Note: DFS and BFS share identical asymptotic time complexity — the difference is purely traversal *strategy*, not speed. Choose based on what the problem is actually asking for, not performance.

DECISION RULE

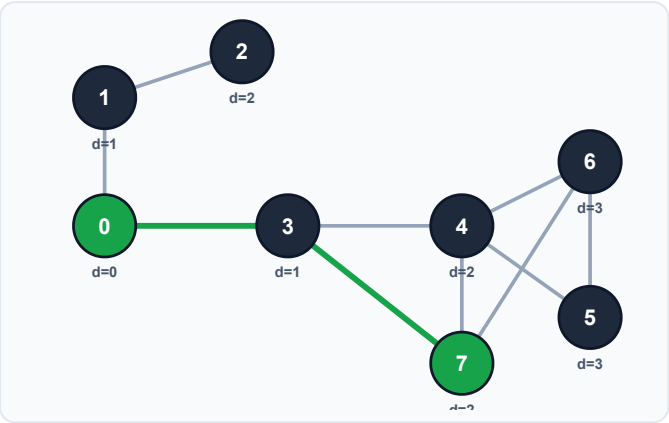
Ask: "do I need the **shortest/fewest-step** answer on an unweighted graph?" → BFS. Ask: "do I need to explore **every possible path**, detect cycles, or order dependencies?" → DFS.

COMMON MISTAKE

Forgetting the outer loop over all vertices (the "driver" in the DFS template) silently breaks on **disconnected** graphs — you'll only traverse the component containing your start vertex and miss the rest entirely.

07 Shortest Path in an Unweighted Graph

Because every edge costs exactly 1 hop, plain **BFS** already finds the shortest path — the first time BFS reaches a vertex, it has done so via the minimum number of edges.



Input: V=8, E=10, S=0, D=7
edges = {0,1},{1,2},{0,3},{3,4},{4,7},{3,7},{6,7},{4,5},{4,6},{5,6}

Output: 0 → 3 → 7

BFS explores level by level. Vertex 7 is first reached in 2 hops via 0→3→7, so that is guaranteed to be the shortest path — even though 0→3→4→7 also reaches 7, it takes 3 hops.

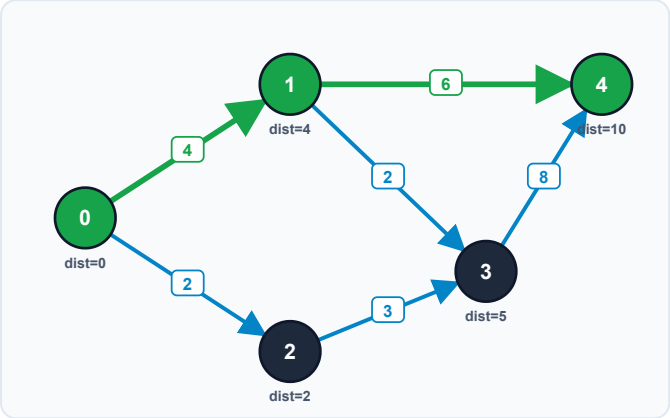
ALGORITHM STEPS

1. Start BFS from the source vertex; mark it visited with distance 0.
2. For each dequeued vertex, visit unvisited neighbors, record `parent[neighbor] = current` and `dist[neighbor] = dist[current] + 1`.
3. Stop early once the destination is dequeued (optional optimization).
4. Reconstruct the path by walking `parent` pointers backward from destination to source.

> 🎯 **FAANG TRIGGER**
> Any "minimum number of steps/moves/transformations" phrasing on an unweighted graph (word ladders, sliding puzzles, grid mazes) → BFS with a `parent` or `dist` map. This pattern alone solves dozens of distinct-sounding LeetCode problems.

07 Dijkstra's Algorithm

Finds the minimum-cost path from a source to **every reachable vertex** in a weighted graph — but only works correctly when all edge weights are **non-negative**.



Source = 0. Shortest path to vertex 4: **0 → 1 → 4**, total cost **4 + 6 = 10** (cheaper than 0→2→3→4 = 2+3+8 = 13).

WORKING

1. Init source distance = 0, all others = ∞.
2. Pick the unvisited vertex with smallest tentative distance (min-heap).
3. Relax all its edges: if `dist[u] + w(u,v) < dist[v]`, update `dist[v]`.
4. Mark the vertex visited (finalized) — never revisit.
5. Repeat until every vertex is processed.

```
import heapq

def dijkstra(graph, src):
    dist = {v: float('inf') for v in graph}
    dist[src] = 0
    pq = [(0, src)]                  # (distance, vertex)
    while pq:
        d, u = heapq.heappop(pq)
        if d > dist[u]:
            continue                 # stale entry, skip
        for v, w in graph[u]:
            if d + w < dist[v]:
                dist[v] = d + w
                heapq.heappush(pq, (dist[v], v))
    return dist
```

FAANG TRIGGER

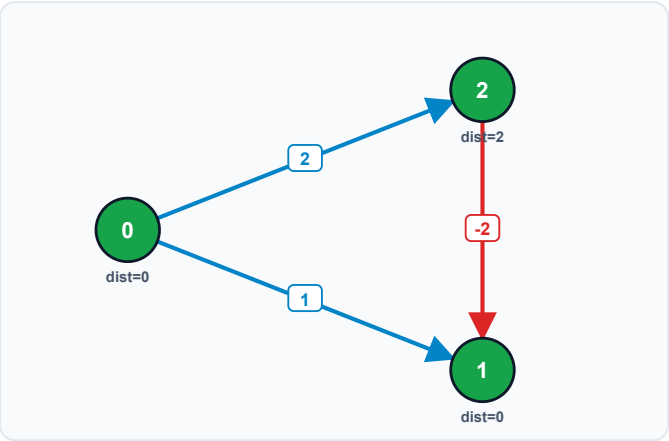
"Cheapest / fastest / minimum cost to reach" with non-negative weights → Dijkstra with a min-heap, O((V+E) log V). Watch for "Network Delay Time" and "Path with Maximum Probability" — both are Dijkstra variants in disguise.

COMMON MISTAKE

Running Dijkstra on a graph with **negative edge weights** silently produces wrong answers — Dijkstra's greedy "finalize on pop" step assumes distances can only increase later. Negative weights break that assumption. Use Bellman-Ford instead.

08 Bellman-Ford Algorithm

Bellman-Ford finds shortest paths from a single source even when edges have **negative weights**, by relaxing *every* edge, **V – 1** times.



Edges: 0→2 (2), 0→1 (1), 2→1 (**–2**). True shortest path 0→1 is **0→2→1 = 2 + (–2) = 0**, not the direct edge (cost 1).

WHY DIJKSTRA FAILS HERE

Dijkstra greedily finalizes vertex 1 the moment it's popped with tentative distance 1 — it never revisits a finalized vertex, so it misses the cheaper route through 2's negative edge. Bellman-Ford has no such "finalize and forget" step.

WORKING

1. Initialize source distance = 0, all others = ∞.
2. Repeat **V – 1** times: relax every edge (u,v,w) — if `dist[u] + w < dist[v]`, update `dist[v]`.
3. Run one extra pass: if any distance still decreases, a **negative-weight cycle** is reachable from the source (no shortest path exists).

```
def bellman_ford(V, edges, src):
    dist = [float('inf')] * V
    dist[src] = 0
    for _ in range(V - 1):
        for u, v, w in edges:
            if dist[u] + w < dist[v]:
                dist[v] = dist[u] + w
    for u, v, w in edges:            # Vth pass: detect negative cycle
        if dist[u] + w < dist[v]:
            raise ValueError("Negative-weight cycle detected")
    return dist
```

🎯 **FAANG TRIGGER**

"Detect arbitrage opportunities", "currency exchange cycles", or any graph explicitly allowing negative weights → Bellman-Ford, O(V·E). Also the standard answer to "how do you detect a negative cycle?".

08 Floyd-Warshall Algorithm

A dynamic-programming algorithm that finds the shortest path between **every pair** of vertices simultaneously, by considering each vertex k as a possible intermediate stop.

Direct-edge distances (before)

	0	1	2	3
0	0	3	∞	7
1	∞	0	1	∞
2	4	∞	0	2
3	5	∞	∞	0

All-pairs shortest distances (after)

	0	1	2	3
0	0	3	4	6
1	5	0	1	3
2	4	7	0	2
3	5	8	9	0

Notice dist[1][0] drops from ∞ to 5 once the algorithm discovers the route **1→2→0** (1 + 4 = 5) — a path that doesn't exist as a direct edge at all. This cascades through every cell as each intermediate vertex k is considered.

RECURRENCE

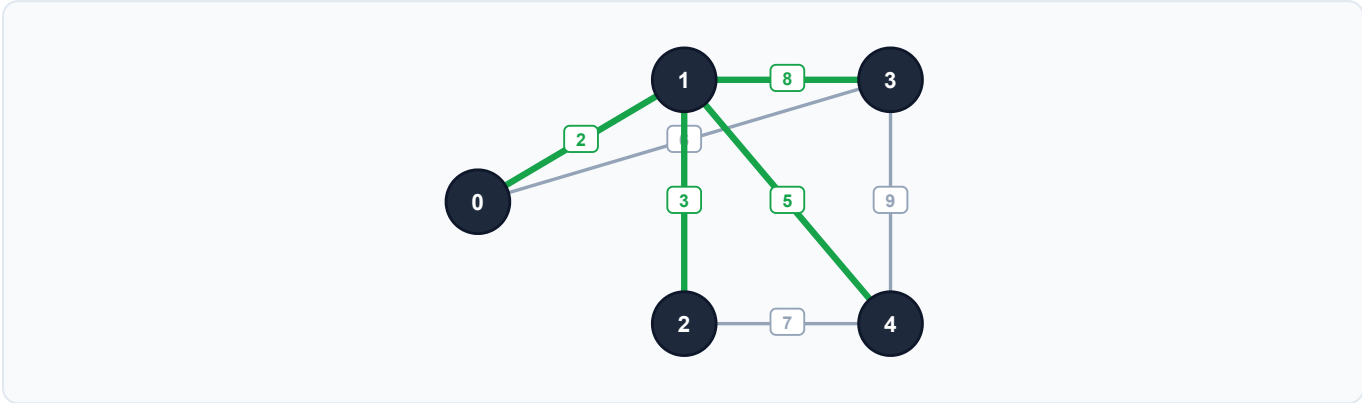
```
for k in range(V):                # intermediate vertex
    for i in range(V):             # source
        for j in range(V):         # destination
            if dist[i][k] + dist[k][j] < dist[i][j]:
                dist[i][j] = dist[i][k] + dist[k][j]
# Time: O(V^3)    Space: O(V^2)
```

> 🎯 **FAANG TRIGGER**
>
> "Shortest path between **all** pairs of vertices" or V is small (≤ ~400-500) → Floyd-Warshall, O(V³). Also used to compute the **transitive closure** of a graph (reachability between every pair).

ALGORITHM	SOURCE(S)	NEGATIVE WEIGHTS	TIME COMPLEXITY
BFS	Single	N/A (unweighted)	O(V + E)
Dijkstra	Single	Not allowed	O((V + E) log V)
Bellman-Ford	Single	Allowed (detects neg. cycles)	O(V · E)
Floyd-Warshall	All pairs	Allowed (detects neg. cycles)	O(V³)

09 Spanning Trees

A **spanning tree** is a subgraph that touches every vertex of a connected graph using the **minimum possible number of edges** — exactly V−1 — while forming no cycles. A single connected graph can have *many* different valid spanning trees.



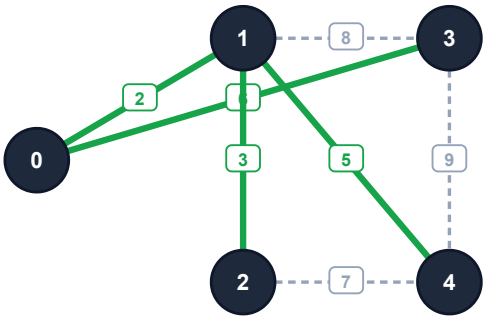
One valid spanning tree of this 5-vertex graph (green edges: 0-1, 1-2, 1-3, 1-4) — 4 edges, all vertices reached, zero cycles. Grey edges are the graph's remaining (unused) connections.

- Contains **all** the vertices of the original graph.
- Has exactly **V – 1** edges.
- Contains **no cycles** — removing any tree edge disconnects it; adding any non-tree edge creates exactly one cycle.

> 🎯 **FAANG TRIGGER**
> "Connect all points/computers/cities with the fewest connections" → you need *a* spanning tree. If cost/distance is involved, you need the cheapest one — the **Minimum Spanning Tree**.

09 Minimum Spanning Tree & Kruskal's Algorithm

The **Minimum Spanning Tree (MST)** is the spanning tree whose edge weights sum to the smallest possible total. **Kruskal's algorithm** builds it greedily: sort all edges by weight, then add each edge unless it would create a cycle (checked with Union-Find).



Final MST (green, weight 2+3+5+6=16). Dashed grey edges were rejected — each would have formed a cycle.

EDGE	WT	KRUSKAL DECISION
0–1	2	✅ Add — merges 0 & 1
1–2	3	✅ Add — merges 1 & 2
1–4	5	✅ Add — merges 1 & 4
0–3	6	✅ Add — merges 0 & 3
2–4	7	❌ Reject — would form a cycle
1–3	8	❌ Reject — would form a cycle
3–4	9	❌ Reject — would form a cycle

WORKING

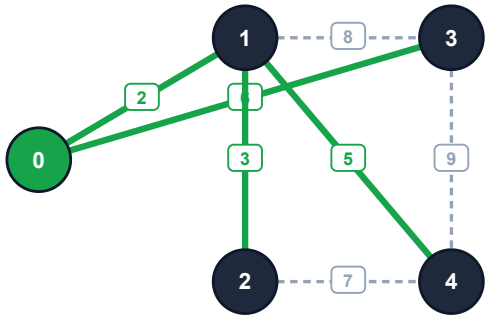
1. Sort all edges in ascending order of weight.
2. Initialize Union-Find with every vertex in its own set.
3. For each edge (u, v) in sorted order: if u and v are in **different** sets, add the edge to the MST and union the sets. Otherwise, skip it (it would form a cycle).
4. Stop once V – 1 edges have been added.

```
def kruskal(V, edges):                # edges: list of (u, v, w)
    parent = list(range(V))
    def find(x):
        while parent[x] != x:
            parent[x] = parent[parent[x]]   # path compression
            x = parent[x]
        return x

    mst_weight, mst_edges = 0, []
    for u, v, w in sorted(edges, key=lambda e: e[2]):
        ru, rv = find(u), find(v)
        if ru != rv:
            parent[ru] = rv
            mst_weight += w
            mst_edges.append((u, v, w))
    return mst_weight, mst_edges
```

> 🎯 **FAANG TRIGGER**
>
> "Minimum cost to connect all cities/points/computers" with explicit edge weights → Kruskal's, O(E log E) dominated by the sort. Pairs perfectly with a **Union-Find** implementation you should have memorized cold.

10 Prim's Algorithm

Prim's algorithm also builds a Minimum Spanning Tree, but grows it **outward from a single starting vertex** — at every step it greedily adds the cheapest edge that connects the growing tree to a new, unvisited vertex.



Starting from vertex 0 (highlighted), Prim's reaches the identical MST as Kruskal's on this graph — same total weight 16.

STEP	VERTEX ADDED	EDGE USED	WT
1	1	0–1	2
2	2	1–2	3
3	4	1–4	5
4	3	0–3	6

At each step, Prim's scans every edge leaving the current tree and picks the single cheapest one leading to a vertex not yet included.

WORKING

1. Pick any vertex as the starting point; mark it "in the tree".
2. Among all edges connecting a tree vertex to a non-tree vertex, pick the minimum-weight one (a min-heap keyed by edge weight makes this fast).
3. Add the new vertex (and edge) to the tree.
4. Repeat until all vertices are included.

```
import heapq

def prim(graph, start):
    visited = {start}
    edges = [(w, start, v) for v, w in graph[start]]
    heapq.heapify(edges)
    mst_weight, mst_edges = 0, []

    while edges and len(visited) < len(graph):
        w, u, v = heapq.heappop(edges)
        if v in visited:
            continue                        # would form a cycle — skip
        visited.add(v)
        mst_weight += w
        mst_edges.append((u, v, w))
        for nxt, nw in graph[v]:
            if nxt not in visited:
                heapq.heappush(edges, (nw, v, nxt))
    return mst_weight, mst_edges
```

FEATURE	KRUSKAL'S	PRIM'S
Strategy	Globally greedy — sorts *all* edges	Locally greedy — grows one tree outward
Core data structure	Union-Find + sorted edge list	Min-heap of frontier edges
Needs a start vertex?	No	Yes
Time complexity	O(E log E)	O(E log V) with binary heap
Best suited for	Sparse graphs, edge-list input	Dense graphs, adjacency-list/matrix input

🎯 **FAANG TRIGGER**

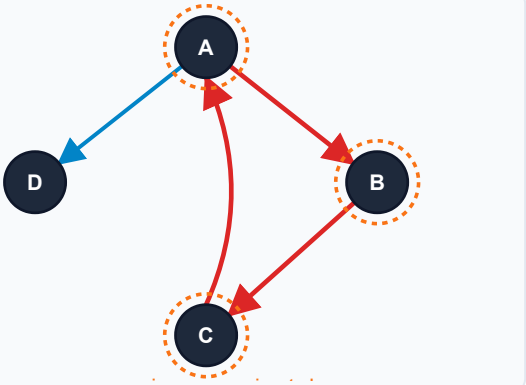
Both algorithms are correct and both are asked about — interviewers often want you to **name both** and justify your pick from graph density. Default to Kruskal's for a sparse edge list, Prim's if you're already holding an adjacency list and a heap.

11 Cycle Detection

The correct technique depends entirely on whether the graph is **directed** or **undirected** — using the wrong one is a very common interview mistake.

Directed Graphs

RECURSION STACK

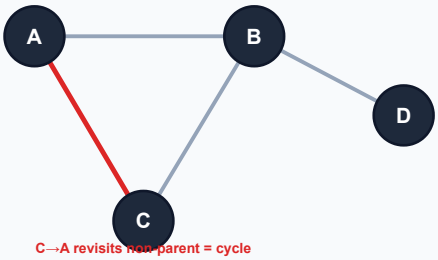


Track two sets during DFS: `visited` (ever visited) and `inStack` (on the *current* recursion path). If DFS reaches a vertex already in `inStack`, that's a **back edge** — a cycle. Remove the vertex from `inStack` when its recursion frame returns.

```
def has_cycle(node, visited, in_stack,
graph):
    visited.add(node)
    in_stack.add(node)
    for nxt in graph[node]:
        if nxt in in_stack:
            return True
# back edge found
        if nxt not in visited and
has_cycle(nxt, visited, in_stack,
graph):
            return True
    in_stack.remove(node)
# done exploring this path
    return False
```

Undirected Graphs

PARENT TRACKING



During DFS, if you reach a vertex that is **already visited and is not your immediate parent**, you've found a cycle. (Simply checking "already visited" isn't enough — every undirected edge looks like it goes "backward" to its own parent.)

```
def has_cycle(node, parent, visited,
graph):
    visited.add(node)
    for nxt in graph[node]:
        if nxt not in visited:
            if has_cycle(nxt, node,
visited, graph):
                return True
        elif nxt != parent:
            return True
# revisited a non-parent
    return False
```

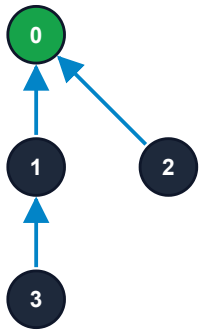
🎯 **FAANG TRIGGER**

"Course Schedule" (can all courses be completed?) is directed-cycle detection. "Redundant Connection" (find the one edge to remove) is undirected — and solved even faster with Union-Find than with DFS.

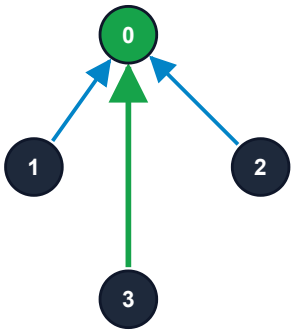
11 Union-Find (Disjoint Set Union)

Union-Find maintains a collection of disjoint sets and answers two questions extremely fast: **"which set is X in?"** (`find`) and **"merge the sets containing X and Y"** (`union`). Every element points to a parent; the **root** of each

tree identifies the set.



Before: find(3) walks 3→1→0 (2 hops) to reach root 0.



After path compression: 3 now points directly to root 0 (green) — future find(3) calls are O(1).

```
class DSU:
    def __init__(self, n):
        self.parent = list(range(n))
        self.rank = [0] * n

    def find(self, x):
        if self.parent[x] != x:
            self.parent[x] = self.find(self.parent[x])    # path compression
        return self.parent[x]

    def union(self, x, y):
        rx, ry = self.find(x), self.find(y)
        if rx == ry:
            return False              # already connected -> this edge creates a cycle
        if self.rank[rx] < self.rank[ry]:
            rx, ry = ry, rx
        self.parent[ry] = rx          # union by rank
        if self.rank[rx] == self.rank[ry]:
            self.rank[rx] += 1
        return True
```

With both **path compression** and **union by rank/size**, every operation runs in **O(α(n))** amortized time — α is the inverse Ackermann function, effectively constant (≤ 4) for any realistic input size.

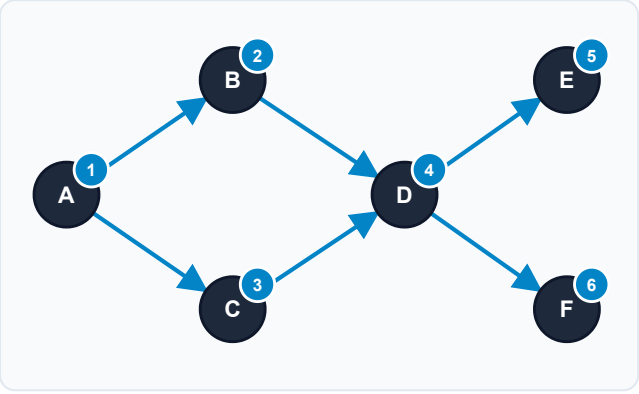
> 🎯 **FAANG TRIGGER**
>
> Union-Find is the fastest tool for: counting connected components ("Number of Provinces"), detecting a cycle while adding edges ("Redundant Connection"), and Kruskal's MST. If a problem adds edges one at a time and asks a connectivity question after each, Union-Find almost certainly beats re-running DFS/BFS.

12 Topological Sort

A topological order is a linear ordering of a DAG's vertices such that for every directed edge (u, v), **u appears before v**. It only exists for DAGs — any cycle makes it impossible.

Kahn's Algorithm

BFS-BASED



Badges show the discovered order: A → B → C → D → E → F

1. Compute **in-degree** for every vertex.
2. Push all in-degree-0 vertices into a queue.
3. Pop a vertex, append to result, decrement its neighbors' in-degree.
4. Any neighbor reaching in-degree 0 joins the queue.
5. If the result has fewer than V vertices at the end, a **cycle** exists.

DFS-based Alternative

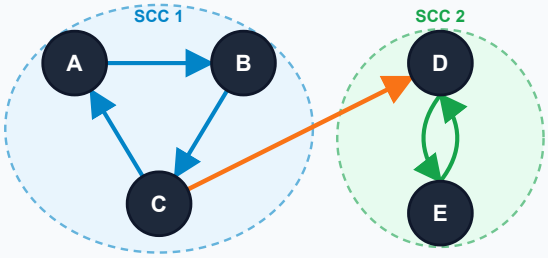
Run DFS from every unvisited vertex; when a vertex's recursion fully finishes (post-order), push it onto a stack. Reversing the stack at the end gives a valid topological order.

```
def topo_sort(V, graph):
    visited, stack = set(), []
    def dfs(u):
        visited.add(u)
        for v in graph[u]:
            if v not in visited:
                dfs(v)
        stack.append(u)           # post-
order: u has no unfinished deps
    for v in range(V):
        if v not in visited:
            dfs(v)
    return stack[::-1]            #
reverse post-order = topo order
```

> 🎯 **FAANG TRIGGER**
>
> "Course Schedule II" (return a valid order), "Alien Dictionary" (derive letter order), "Build order" for a package manager — all are topological sort with a twist on how the DAG is built.

12 Strongly Connected Components (SCC)

A **strongly connected component** is a maximal group of vertices where *every* vertex can reach *every other* vertex by following directed edges. The bridge edge C→D below is one-directional, so A/B/C and D/E remain separate SCCs even though the whole graph is connected.



Two SCCs: {A, B, C} (blue) and {D, E} (green), joined by a one-way bridge C→D that is not part of either cycle.

KOSARAJU'S ALGORITHM

1. Run DFS on the original graph, pushing vertices to a stack in **finish order**.
2. Reverse (transpose) every edge in the graph.
3. Pop vertices from the stack; for each unvisited one, DFS on the **transposed** graph — everything reached in that DFS is one SCC.

TARJAN'S ALGORITHM

A single-pass DFS that tracks a **discovery time** and a **low-link value** (the lowest discovery time reachable) for each vertex, using an explicit stack. A vertex whose low-link equals its own discovery time is the root of an SCC — pop the stack down to it to emit that component.

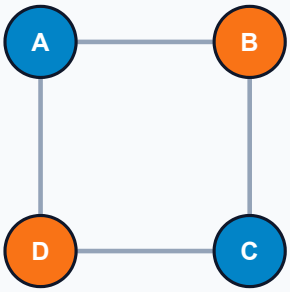
🎯 FAANG TRIGGER

SCC questions show up as "find groups of mutually reachable servers/accounts" or as a building block for condensing a graph into a DAG of components (used in 2-SAT and dependency-resolution problems). Know Kosaraju's conceptually — Tarjan's is the follow-up "can you do it in one pass?".

13 Bipartite Check (2-Coloring)

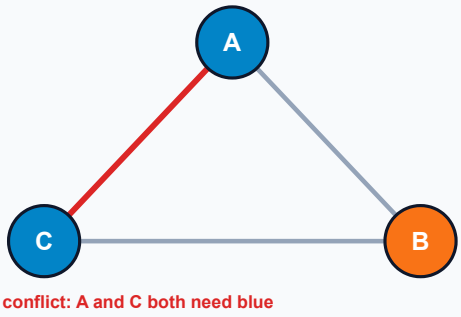
A graph is bipartite **if and only if it contains no odd-length cycle**. The standard test: try to color every vertex with one of two colors such that no edge connects two same-colored vertices, using BFS or DFS.

Bipartite ✅



4-cycle A-B-C-D-A (even length) colors perfectly: blue, orange, blue, orange — no conflicts.

Not Bipartite ❌



Odd 3-cycle A-B-C-A: after A=blue, B=orange, C must differ from B (blue) — but C is also adjacent to A, which is already blue. Conflict ⇒ not bipartite.

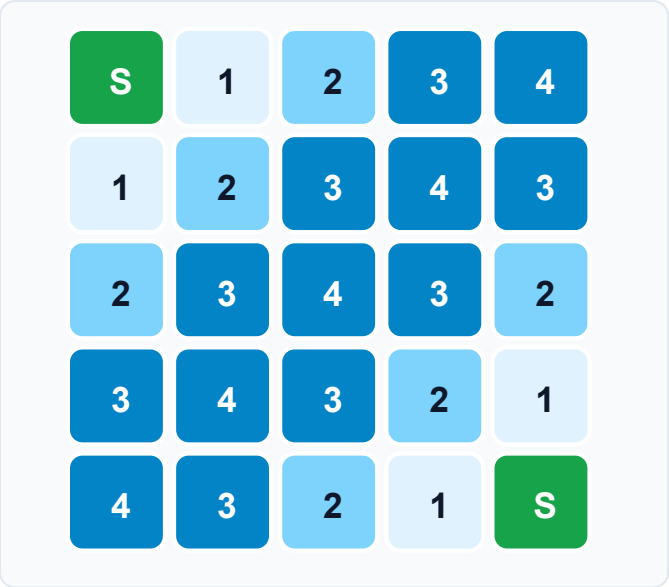
```
from collections import deque

def is_bipartite(graph, n):
    color = [-1] * n
    for start in range(n):
        if color[start] != -1:
            continue
        color[start] = 0
        q = deque([start])
        while q:
            u = q.popleft()
            for v in graph[u]:
                if color[v] == -1:
                    color[v] = 1 - color[u]     # opposite color
                    q.append(v)
                elif color[v] == color[u]:
                    return False                 # same color neighbors -> conflict
    return True
```

> 🎯 **FAANG TRIGGER**
> "Is Graph Bipartite?", "Possible Bipartition" (can people be split into 2 groups given dislike-pairs?), and coloring-constraint problems all reduce directly to this exact BFS 2-coloring template.

13 Multi-Source BFS & Grids as Graphs

A 2D grid is just a graph in disguise: each cell is a vertex, with edges to its (usually 4) orthogonal neighbors. When a problem starts from **multiple cells at once** ("rotting oranges", "distance to nearest exit"), push *all* sources into

the BFS queue before the first step — this finds the true minimum distance from **any** source in a single pass.



Two sources (green "S" at top-left and bottom-right) spread simultaneously. Every other cell shows the number of steps to reach the **nearest** source — computed in one BFS pass starting with both sources already in the queue.

> 🎯 **FAANG TRIGGER**
>
> Whenever a grid problem has more than one starting point ("all rotten oranges", "all fire cells", "nearest of several exits") → seed the BFS queue with **every** source at distance 0 up front, rather than running separate BFS per source (which is asymptotically worse).

```
def multi_source_bfs(grid, sources):
    from collections import deque
    R, C = len(grid), len(grid[0])
    dist = [[-1]*C for _ in range(R)]
    q = deque()
    for r, c in sources:
        dist[r][c] = 0
        q.append((r, c))
    while q:
        r, c = q.popleft()
        for dr, dc in ((-1,0),(1,0),(0,-1),(0,1)):
            nr, nc = r+dr, c+dc
            if 0<=nr<R and 0<=nc<C and dist[nr][nc]==-1:
                dist[nr][nc] = dist[r][c] + 1
                q.append((nr, nc))
    return dist
```

Number of Islands — DFS/BFS flood-fill from every unvisited land cell, counting how many traversals you start (component counting on an implicit grid graph).

Rotting Oranges — multi-source BFS; answer is the max distance value reached (time for the last orange to rot).

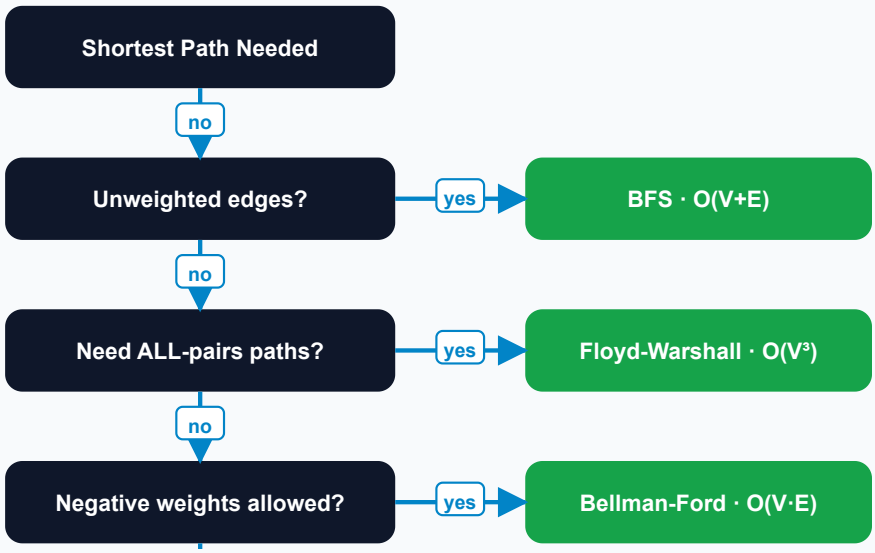
Word Search — DFS with backtracking on the grid graph, marking/unmarking cells as "visited" along the current path.

Shortest Path in Binary Matrix — plain BFS (8-directional) from top-left to bottom-right, obstacles as blocked vertices.

14 FAANG Pattern Cheat Sheet

The night before an interview, don't re-read every algorithm — re-read this page. Every row is a phrase pattern interviewers use and the technique it's really asking for.

Shortest-Path Decision Tree



IF THE PROBLEM SAYS...	...REACH FOR
"Fewest steps / minimum moves", unweighted	**Plain BFS**
"Cheapest / fastest", non-negative weights	**Dijkstra**
Weights can be negative	**Bellman-Ford**
Need shortest path between every pair	**Floyd-Warshall**
"Connect all cities/points" with min total cost	**Kruskal's or Prim's MST**
"Prerequisite / build order / dependency"	**Topological Sort**
"Can this ever loop forever?" (directed)	**DFS + recursion stack**
"Redundant connection" / cycle in undirected graph	**Union-Find (DSU)**

IF THE PROBLEM SAYS...	...REACH FOR
"Count islands / provinces / groups"	**Component counting: DFS/BFS or Union-Find**
"Split into two groups" / dislike-pairs	**Bipartite check (BFS 2-coloring)**
Multiple starting points spreading together	**Multi-source BFS**
"Does any path exist / explore every path"	**DFS**
"Mutually reachable accounts/servers"	**SCC — Kosaraju's / Tarjan's**
"Clone / deep-copy this graph"	**DFS or BFS + hash map of original→copy**
Word transformation sequence (word ladder)	**BFS over an implicit word graph**
Grid/matrix with neighbor moves	**Treat cells as vertices; 4- or 8-directional edges**

Master Complexity Recap

ALGORITHM	TIME	SPACE
DFS / BFS (adjacency list)	O(V + E)	O(V)
Dijkstra (binary heap)	O((V + E) log V)	O(V)
Bellman-Ford	O(V · E)	O(V)
Floyd-Warshall	$O(V^3)$	$O(V^2)$
Kruskal's MST	O(E log E)	O(V)
Prim's MST (binary heap)	O(E log V)	O(V)
Union-Find (path compression + rank)	O(α(V)) ≈ O(1) amortized	O(V)
Topological Sort (Kahn's / DFS)	O(V + E)	O(V)
Kosaraju's / Tarjan's (SCC)	O(V + E)	O(V)
Bipartite check (2-coloring)	O(V + E)	O(V)

INTERVIEW DAY ADVICE

Say your classification out loud before coding: *"This is an undirected, unweighted, possibly disconnected graph — I'll use BFS from every unvisited vertex to count components."* Naming the graph's properties signals seniority before you write a single line.

15 50 Curated Practice Problems

Every technique from this guide, drilled through 50 real interview-style problems — ordered so each one introduces exactly one new wrinkle on a pattern you already know.

Easy

WARM-UPS — BUILD PATTERN RECOGNITION

EASY **Find if Path Exists in Graph** · BFS/DFS reachability
Check whether a path connects two given nodes in an undirected graph.

EASY **Number of Islands** · Component counting
DFS/BFS flood-fill from every unvisited land cell in a grid; count how many traversals you start.

EASY **Flood Fill** · Grid DFS/BFS
Recolor the connected region of matching-color pixels starting from one cell.

EASY **Island Perimeter** · Grid traversal
For each land cell, add 4 minus the number of land neighbors to a running perimeter total.

EASY **Find Center of Star Graph** · Degree observation
The vertex shared by the first two given edges must be the star's center.

EASY **Employee Importance** · Tree-shaped DFS/BFS
Sum an employee's importance with that of every subordinate reached via DFS/BFS.

EASY **Keys and Rooms** · Reachability
DFS/BFS from room 0; the rooms are 'unlocked' if every node is reachable.

EASY **Number of Provinces** · Component counting
DFS/BFS or Union-Find over an adjacency-matrix-style 'friend circle' graph.

EASY **Minimum Number of Vertices to Reach All Nodes** · In-degree scan
In a DAG, exactly the vertices with in-degree 0 are required starting points.

EASY **Find the Town Judge** · In/out-degree
The judge has in-degree V−1 and out-degree 0 — a pure degree-counting problem.

EASY **N-ary Tree Preorder Traversal** · DFS on a tree
Standard DFS practice on a tree, which is simply an acyclic connected graph.

EASY **All Paths From Source to Target** · DFS backtracking
Enumerate every path from node 0 to node n−1 in a DAG via backtracking DFS.

EASY **Maximum Depth of N-ary Tree** · BFS/DFS depth
Level-order BFS (or DFS with depth tracking) on a tree-shaped graph.

EASY **Binary Tree Level Order Traversal** · BFS levels
Classic queue-based BFS producing one output list per level.

EASY **Path Sum** · DFS root-to-leaf
Depth-first search accumulating a running sum down each root-to-leaf path.

15 Medium Problems

Medium

CORE INTERVIEW BAR

MED **Clone Graph** · DFS/BFS + hash map
Traverse the graph while mapping original node → cloned node to rebuild all edges.

MED **Course Schedule** · Directed cycle detection
Model prerequisites as directed edges; a valid schedule exists iff there's no cycle.

MED **Course Schedule II** · Topological sort
Kahn's BFS (in-degree queue) or DFS post-order to output one valid course order.

MED **Number of Connected Components in an Undirected Graph** · Component counting
DFS/BFS or Union-Find; count how many disjoint sets remain.

MED **Graph Valid Tree** · Tree validation
A graph is a tree iff it has exactly V−1 edges and is fully connected (no cycle).

MED **Redundant Connection** · Union-Find
Add edges one by one; the first edge joining two already-connected nodes is the answer.

MED **Is Graph Bipartite?** · BFS/DFS 2-coloring
Try to 2-color every component; a conflict means the graph is not bipartite.

MED **Possible Bipartition** · Bipartite modeling
Model 'dislike' pairs as edges, then run the standard bipartite check.

MED **Rotting Oranges** · Multi-source BFS
Seed the BFS queue with every rotten orange at once; answer is the max distance reached.

MED **Shortest Path in Binary Matrix** · 8-directional BFS
Plain BFS from top-left to bottom-right treating 0-cells as traversable vertices.

MED **Word Ladder** · Implicit-graph BFS
Each word is a vertex; edges connect words differing by one letter — BFS finds the shortest transformation.

MED **Network Delay Time** · Dijkstra
Single-source shortest path on a directed weighted graph; answer is the max distance to any node.

MED **Cheapest Flights Within K Stops** · Bounded Bellman-Ford
Relax edges at most K+1 times (or level-limited BFS) to respect the stop limit.

MED **Path with Maximum Probability** · Dijkstra variant
Same as Dijkstra but maximize a product of edge probabilities instead of minimizing a sum.

MED **Evaluate Division** · Weighted graph DFS/BFS
Build a weighted graph from ratios; answer each query with a DFS/BFS product-of-weights walk.

MED **Reorder Routes to Make All Paths Lead to City Zero** · Directed-edge tracking
DFS the underlying undirected graph, counting edges that point 'away' from the root.

MED **Minimum Height Trees** · Leaf peeling
Repeatedly remove degree-1 leaves layer by layer; the last 1-2 nodes left are the centroids.

MED **Find Eventual Safe States** · 3-color DFS cycle detection
White/gray/black DFS marking finds nodes that never lead into a cycle.

MED **Accounts Merge** · **Union-Find**

Union accounts that share any email, then group and sort emails per root.

MED **As Far from Land as Possible** · **Multi-source BFS**

Seed BFS with every land cell; the answer is the maximum distance recorded on water.

15 Hard Problems

Hard

STAFF+ / ONSITE CLOSERS

HARD **Word Ladder II** · **BFS + DFS reconstruction**
BFS to find shortest distance layers, then DFS backtrack to build every shortest path.

HARD **Alien Dictionary** · **Build-a-DAG + topo sort**
Derive character-ordering edges from adjacent words, then topologically sort the alphabet.

HARD **Swim in Rising Water** · **Binary search + BFS / modified Dijkstra**
Find the minimum time threshold such that a path of low-enough cells connects the corners.

HARD **Bus Routes** · **Multi-level BFS**
Treat routes (not stops) as graph vertices; BFS over which routes are reachable from each other.

HARD **Minimum Cost to Make at Least One Valid Path in a Grid** · **0-1 BFS**
Deque-based BFS: 0 cost to follow the arrow, 1 cost to redirect — front/back push simulates a priority queue cheaply.

HARD **Critical Connections in a Network** · **Tarjan's bridge-finding**
Track discovery time and low-link values; an edge is a bridge if the child's low-link exceeds the parent's discovery time.

HARD **Reconstruct Itinerary** · **Eulerian path (Hierholzer's)**
Greedily DFS smallest-lexical edges first, adding to the itinerary in post-order, then reverse.

HARD **Sliding Puzzle** · **State-space BFS**
Each board configuration is a vertex; BFS over neighboring swaps finds the minimum moves to solve.

HARD **Number of Islands II** · **Online Union-Find**
Add land cells one at a time, union with land neighbors, and track the live component count after each addition.

HARD **Optimize Water Distribution in a Village** · **MST with virtual source**
Model well costs as edges from a virtual node 0, then run Kruskal's/Prim's over the combined graph.

HARD **Parallel Courses III** · **DAG longest path**
Topologically process the DAG while tracking the longest (slowest) prerequisite chain time per node.

HARD **Shortest Path Visiting All Nodes** · **Bitmask BFS**
State = (current node, visited-set bitmask); BFS over this expanded state graph finds the minimum tour.

HARD **Find Critical and Pseudo-Critical Edges in MST** · **Kruskal's force include/exclude**
Re-run Kruskal's forcing each edge in or out to see if the MST weight changes.

HARD **Cut Off Trees for Golf Event** · **Repeated BFS**
Sort trees by height, then BFS between consecutive targets summing the step counts.

HARD **Making A Large Island** · **Flood-fill + union simulation**
Label each island's component and size, then test flipping every 0 by summing its distinct neighboring island sizes.

✓ Your Study Plan

Graphs reward spaced repetition more than any other DSA topic — the same six or seven templates (BFS, DFS, Dijkstra, Union-Find, Topological Sort, MST) recombine into hundreds of distinct-looking problems.

Week 1 — FOUNDATIONS

Sections 01–05. Memorize every graph type and representation until you can classify any prompt in under 10 seconds.

Week 2 — CORE ALGORITHMS

Sections 06–12. Hand-write DFS, BFS, Dijkstra, Kruskal's, Prim's, and Union-Find from memory — no looking, no copy-paste.

Week 3 — DRILL & SPEED

Sections 13–15. Work all 50 curated problems, easy → hard, timing yourself against a 25–35 minute onsite budget.

The Only Checklist You Need Mid-Interview

1. **Classify the graph out loud:** directed/undirected, weighted/unweighted, likely connected or not.
2. **Pick a representation:** adjacency list unless you have a specific reason for a matrix.
3. **Match the ask to section 14's cheat sheet:** shortest path? spanning tree? components? ordering?
4. **State the complexity** before you finish coding — interviewers are listening for this as much as correctness.
5. **Test on a disconnected / empty / single-node input** out loud — the classic edge cases graders check first.

FINAL WORD

You now have every graph type, every representation, every core algorithm, a rapid-recall cheat sheet, and 50 problems to prove it to yourself. That's the whole syllabus — the only variable left is repetition. Good luck.